

Evaluation AI

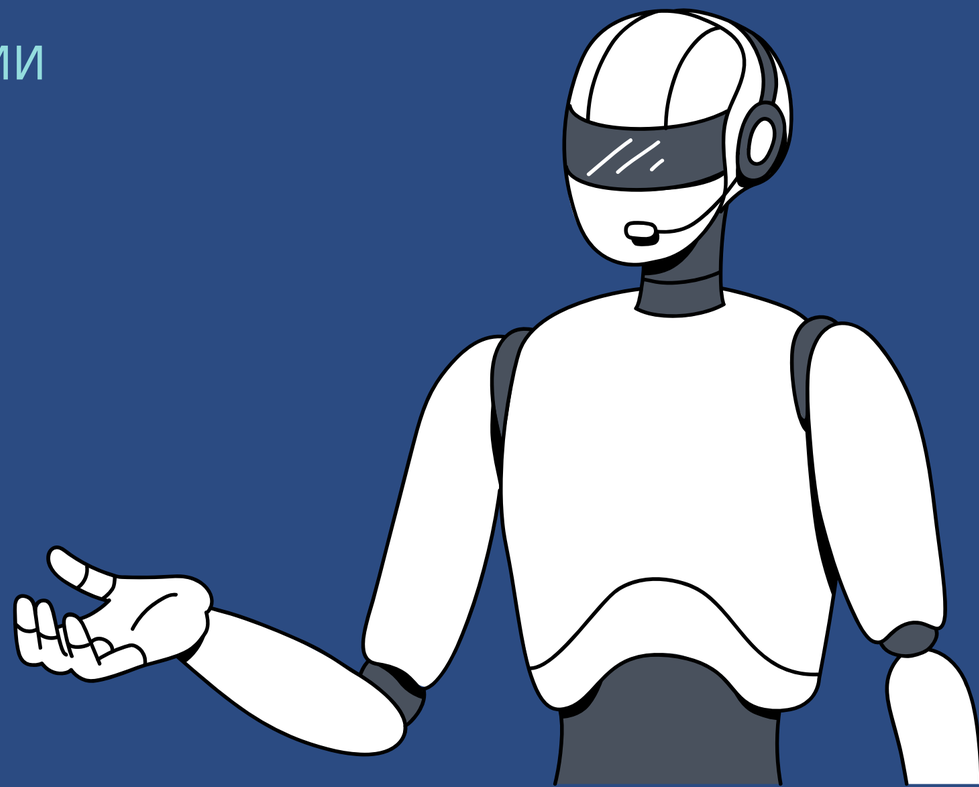
Проектирование датасета для оценки ИИ Evaluation RAG систем

Автор курса Мешков Александр

Лекция 4: Создание датасета и оценка RAG систем

ПЛАН ЛЕКЦИИ:

1. Что такое кастомный датасет и зачем он нужен
2. Критерии создания датасета для оценки прикладного ИИ
3. Golden Dataset
4. Что такое RAG
5. Архитектура и типы RAG
6. Метрики оценки RAG систем



Два типа evaluation

Академическая оценка (оценка ИИ модели)

LLM Модели (Автоматические метрики NLP, Бенчмарки, LLM Arena)

COMPLETED



Прикладная оценка (оценка ИИ системы)

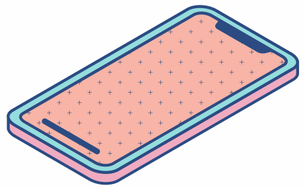
Retrieval-augmented generation (RAG)

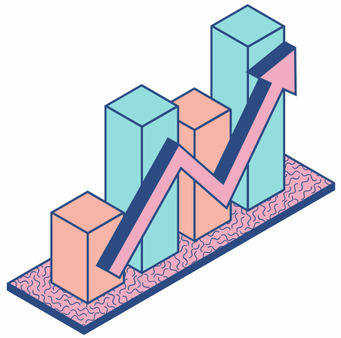
ИИ агенты

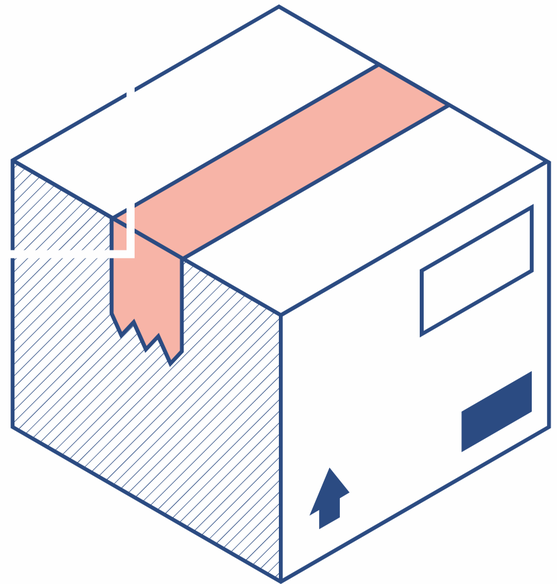
Чат-боты и мультиходовые агенты

AI Teammates



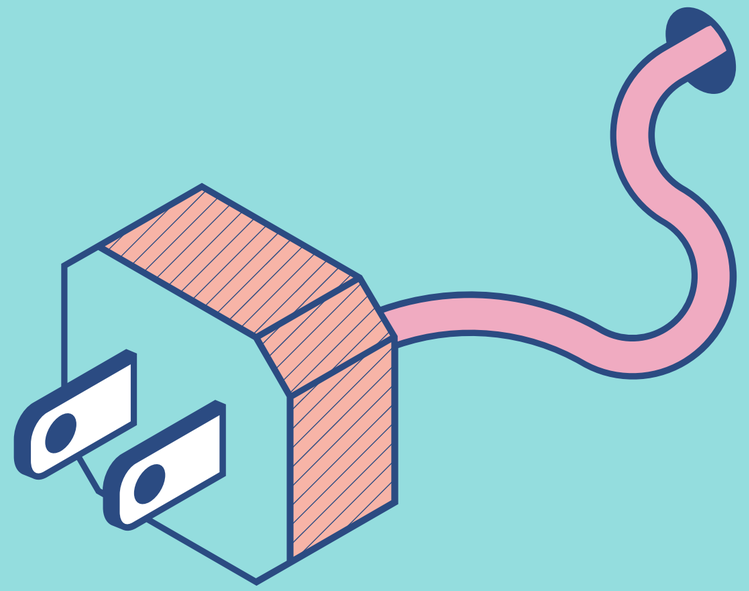






Сложность оценки ИИ систем на базе LLM

Как оценивать, если нет ожидаемого результата?
Где границы работы ИИ?
Что есть хороший показатель качества?
Когда тестирования достаточно?



Создание кастомного датасета для evaluation

Создание кастомного датасета является критически важным навыком для тестировщиков LLM, поскольку готовые бенчмарки часто не покрывают специфические требования бизнеса или уникальные сценарии использования.

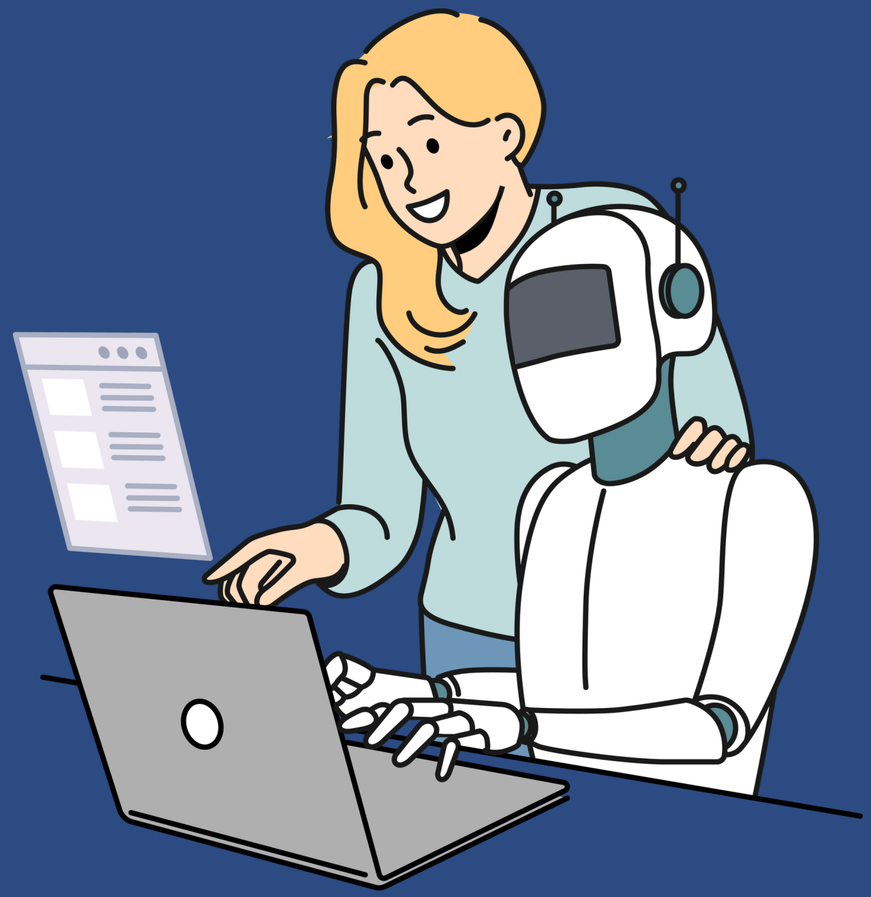
Критерии создания датасета включают реалистичность сценариев, которые отражают реальное поведение пользователей, комплексное покрытие различных аспектов функциональности ИИ системы, вариативность сложности запросов, включение граничных случаев и необычных, но валидных запросов, а также фокус на частых паттернах взаимодействия.

Автоматическая генерация позволяет создавать большие объемы тестовых данных с помощью специализированных промптов, которые учитывают контекст AI, форматы входных и выходных данных, типы тестов, характеристики вопросов и требования к качеству.

ЭТАПЫ СОЗДАНИЯ КАСТОМНОГО ДАТАСЕТА

Основные стадии:

1. **Понимание задачи** - специфичные use cases и требования к ИИ системе
2. **Определение доменов использования** - независимые области для оценки
3. **Сбор данных** - реальные кейсы использования ИИ системы (если есть)
4. **Формирование примерного ожидаемого результата** - (если возможно)
5. **Проверка качества данных** - верификация с помощью экспертов в компании



КРИТЕРИИ ГЕНЕРАЦИИ КАСТОМНОГО ДАТАСЕТА

1. Контекст агента и области применения
2. Формат входных данных, принципы дизайна данных
3. ! Направление тестирования
4. ! Типы запросов
5. Параметры запросов
   - Длина запросов
   - Открытость вопросов
   - Язык
6. Вариативность сложности
   - Простые задачи
   - Задачи средней сложности
   - Сложные задачи
7. Формат выходных данных



Направления тестирования прикладного ИИ (типы датасетов)

Happy path (позитивное тестирование)

Edge Cases (граниченые ситуации и случаи)

Negative checks (негативное тестирование)

Важно! Edge cases могут быть как частью happy path данных, так и частью негативных проверок

Metamorphoric relations (метаморфическое тестирование)

Consistency (тестирование воспроизвоимости поведения)

Safety (тестирование безопасности)

Robustness

Bias/Toxicity/Social (социальные аспекты)

МЕТАМОРФИЧЕСКИЕ ОТНОШЕНИЯ (ПРОВЕРКА НАДЕЖНОСТИ)

MR1: Замена синонимами. Проверка того, умеет ли модель обрабатывать синонимы и возвращать релевантный результат.
MR2: Перефразировка. Оценка способности модели корректно отвечать на перефразированные (идентично измененные) входные данные.
MR3: Добавление контекста. Проверка влияния дополнительного контекста на точность извлечения и генерации.
MR4: Негация. Оценка того, как модель справляется с отрицанием в запросах.
MR5: Сдвиг фокуса. Проверка, удерживает ли модель правильный фокус в запросе и не теряет ли целевую информацию.
MR6: Составной вопрос. Оценка качества работы с составными вопросами и способностью давать точные ответы на все части.
MR7: Двусмысленность. Проверка, может ли модель правильно разрешать неоднозначность и выбирать верную сущность.
MR8: Частичная информация. Оценка способности модели работать с урезанными запросами и всё равно выдавать точный ответ.

ПРИМЕРЫ ТИПОВ ТЕСТОВ ДЛЯ ГЕНЕРАЦИИ

Вариант 1: Категории по типу когнитивной задачи

```
test_types = [
    "factual",      # Проверка фактов, дат, определений, цифр
    "reasoning",    # Логические рассуждения, цепочка вывода
    "instructional",  # Пошаговые инструкции, алгоритмы действий
    "creative",     # Генерация текста, идей, историй
    "comparative",  # Сравнение предметов, технологий, подходов
    "analytical",   # Анализ ситуации, кода, логов, данных
    "diagnostic",   # Поиск ошибок, багов, уязвимостей
    "evaluative"    # Оценка — что лучше, правильнее, эффективнее
]
```

Вариант 2: Категории, ближе к диалоговым сценариям

```
test_types = [
    "question_answering",  # Ответ на конкретный вопрос
    "summarization",     # Сжатие текста / вывод сути
    "rewriting",       # Переформулировка / упрощение
    "classification",    # Определение категории, типа, состояния
    "generation",      # Создание оригинального контента
    "translation",     # Перевод
    "extraction",      # Извлечение информации
    "debugging"        # Анализ и исправление ошибок
]
```

Вариант 3: Для обучения/валидации LLM на QA и ассистивных задачах

```
test_types = [
    "knowledge_retrieval",  # Проверка знаний
    "reasoning_chain",    # Пошаговое объяснение вывода
    "tool_use",       # Использование внешних инструментов (например, код, SQL)
    "code_generation",    # Написание кода
    "error_explanation",  # Объяснение, почему что-то не работает
    "problem_solving",    # Решение задач
    "planning",       # Планирование действий / проектов
    "open_discussion"     # Развернутый диалог без точного ответа
]
```

Golden Datasets

Golden датасеты - это эталонные набор данных (например, пользовательских запросов) с заранее проверенными правильными ответами (expected answer).

Зачем он нужен:

- Проверка регрессий
- Стабильная точка контроля
- Автоматизация.

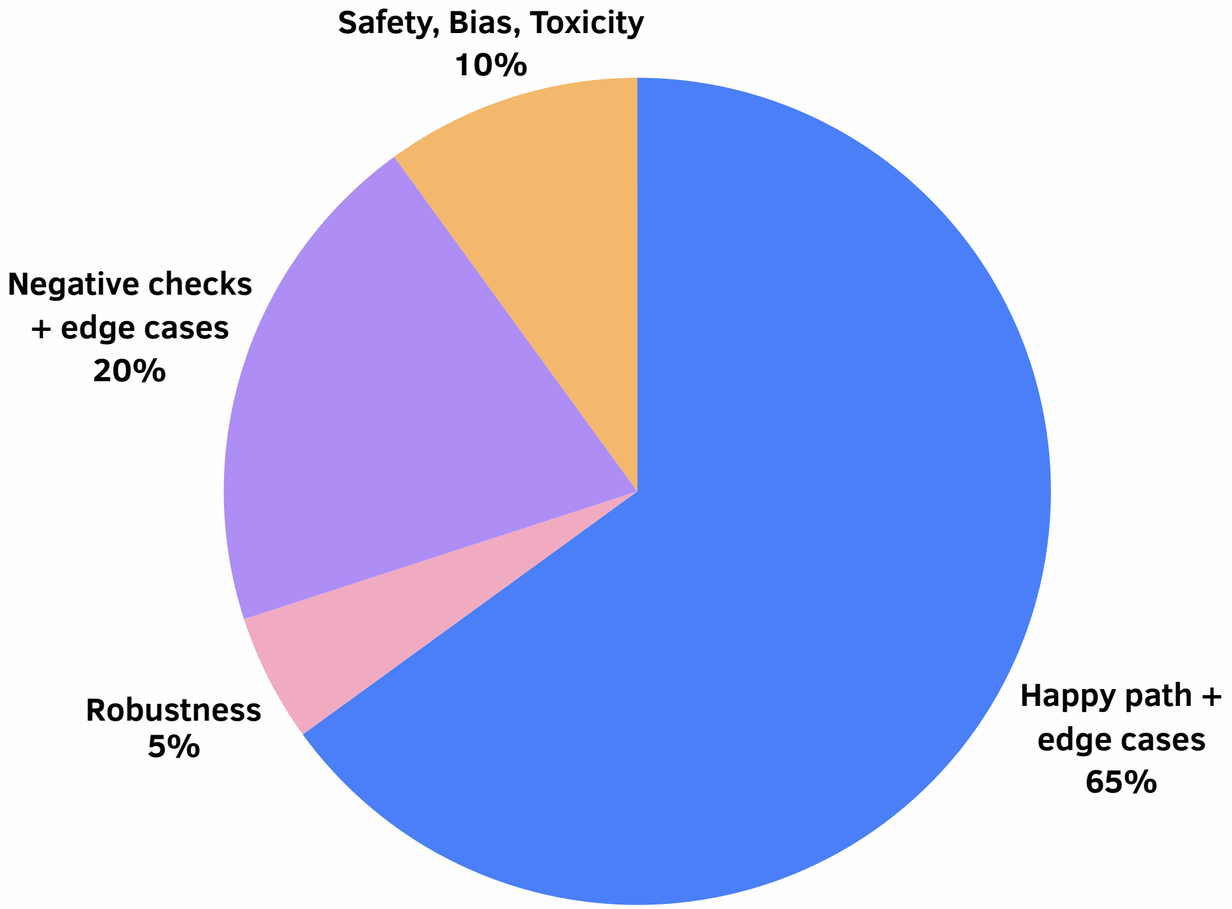
Как правильно его спроектировать:

1. Подбирать репрезентативные кейсы.
2. Добавлять edge cases.
3. Фиксировать корректные ответы.
4. Поддерживать версионность.
5. Встраивать в пайплайн.

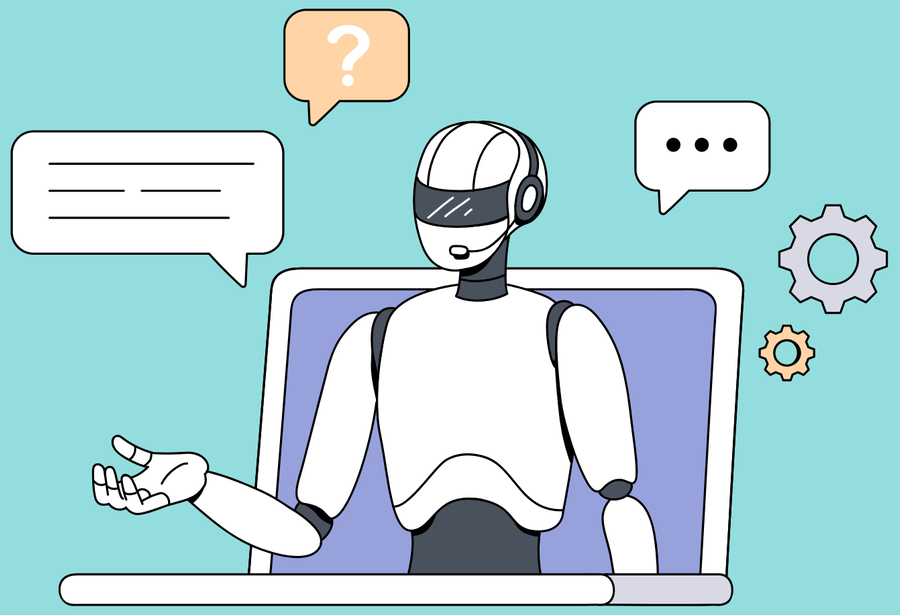
По сути, golden датасет должен говорить нам о том, что новые версии AI решения не ломают логику работы. Это та самая контрольная работа, которую AI продукт сдает перед каждым релизом.

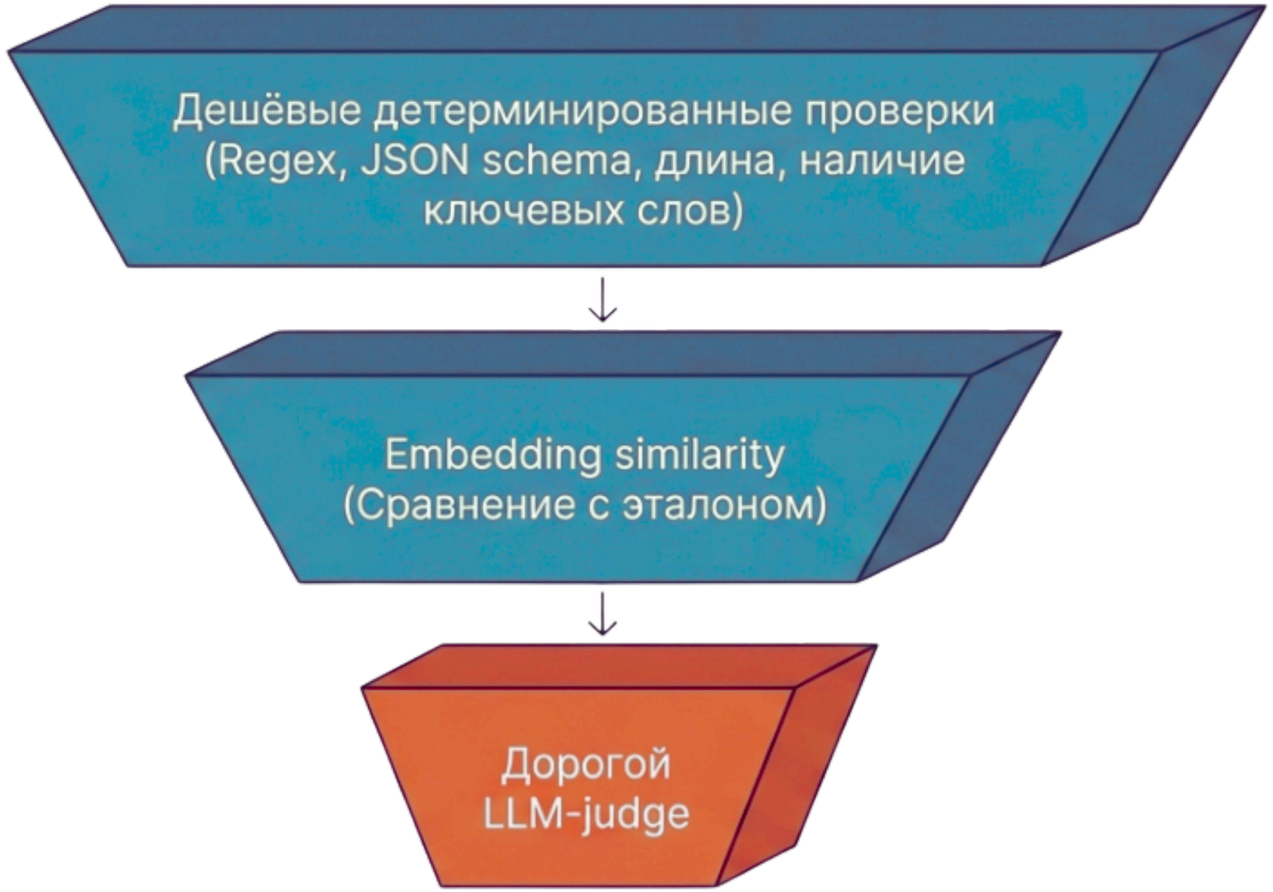
Структура golden datasets



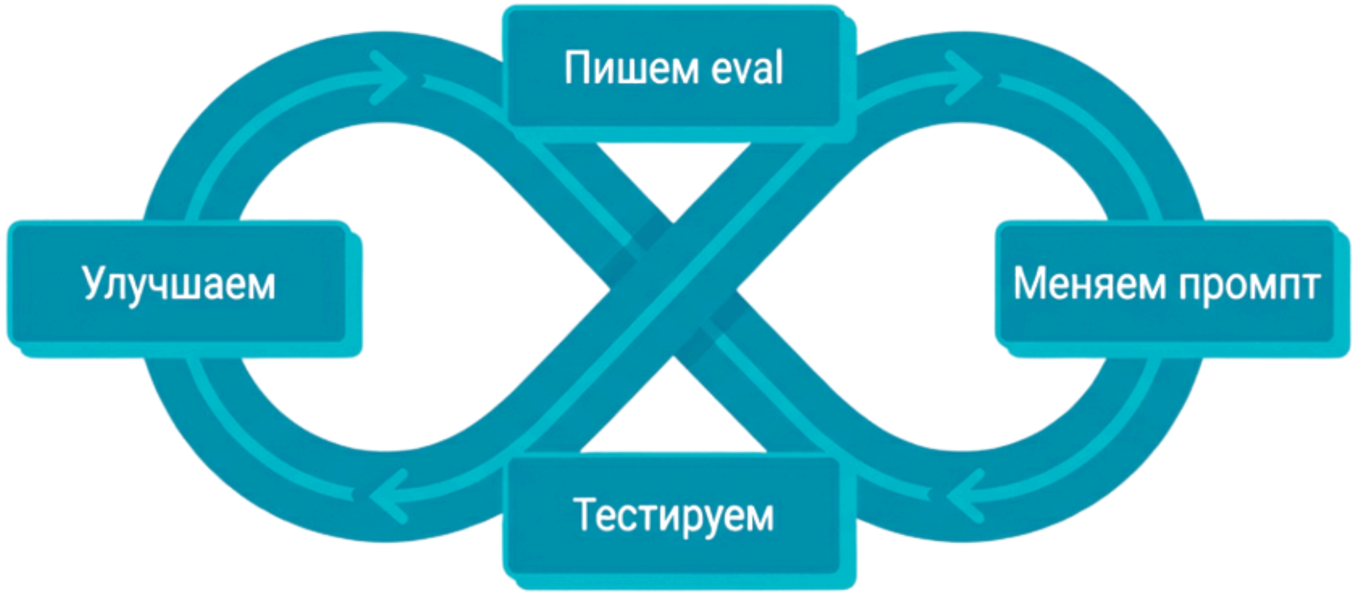


Eval Pipeline

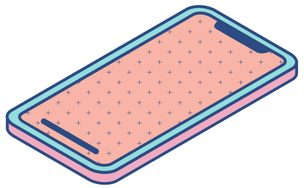


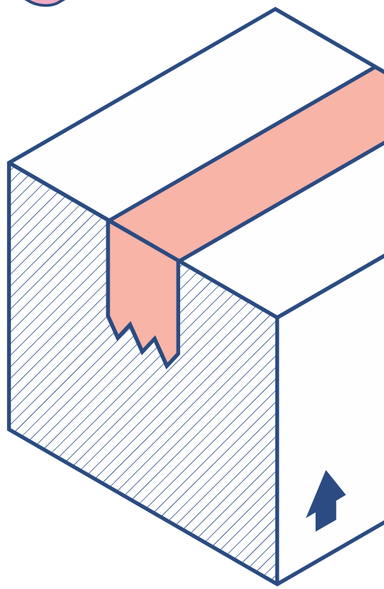


EVAL DRIVEN DEVELOPMENT







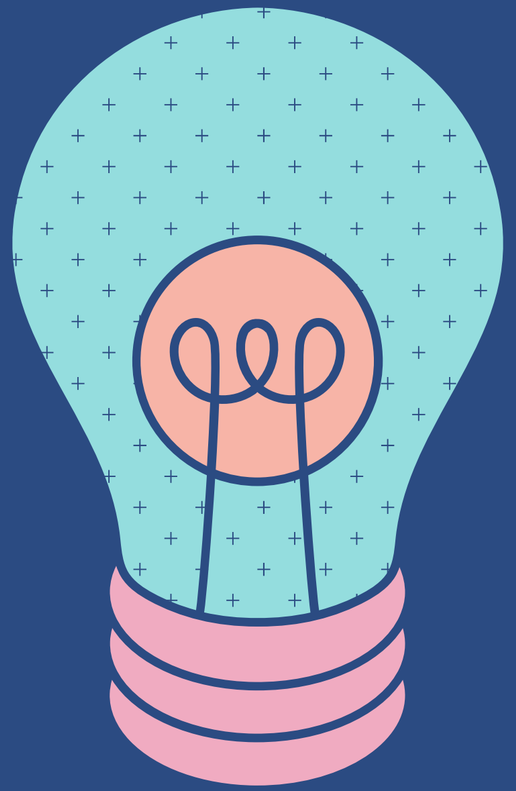


Аналогия с классическим TDD

Сначала создаем датасет, потом пишем промпты и разрабатываем программу

Проверка регрессии на каждое изменение

RAG (Retrieval-Augmented Generation) — это архитектурный подход в искусственном интеллекте, который объединяет возможности поиска релевантной информации (retrieval) с генерацией ответов на основе найденных данных (generation). Эта технология позволяет языковым моделям получать доступ к актуальной и специфической информации из внешних источников знаний в процессе формирования ответа.



Архитектура RAG



Компонент поиска (Retrieval Component) отвечает за нахождение релевантных документов или фрагментов текста из базы знаний. Этот процесс включает несколько этапов:

- Индексация документов: все документы в базе знаний преобразуются в векторные представления (embeddings) с помощью специальных моделей энкодинга. Эти векторы захватывают семантический смысл текста и позволяют находить схожие по содержанию фрагменты.
- Обработка запроса: пользовательский вопрос также преобразуется в векторное представление с помощью той же модели энкодинга.
- Семантический поиск: система находит наиболее семантически близкие фрагменты документов, сравнивая векторы запроса и документов. Обычно используются метрики типа косинусного сходства.

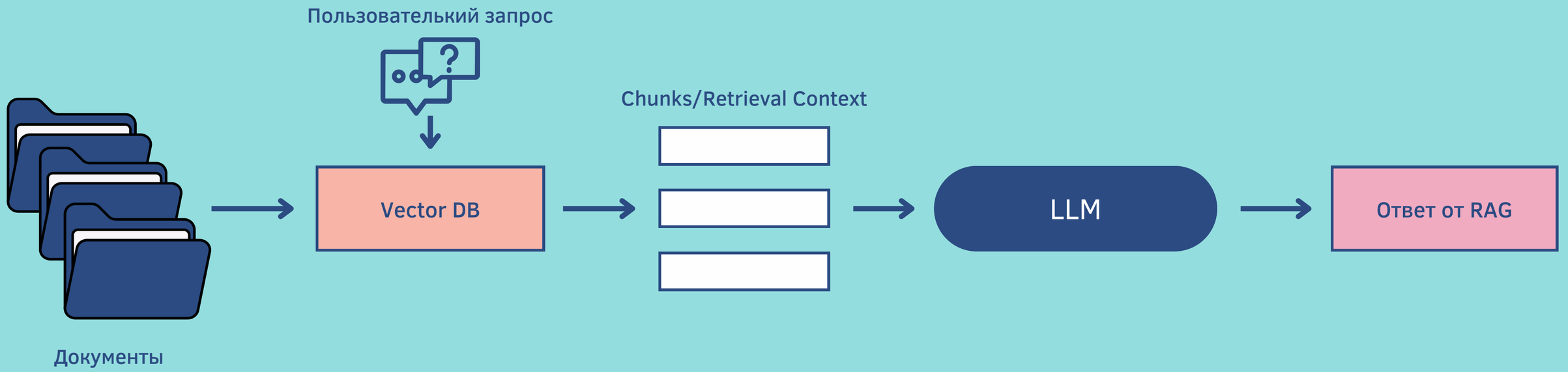
Архитектура RAG



Компонент генерации (Generation Component) использует найденную информацию для создания ответа:

- Контекстуализация: найденные релевантные фрагменты объединяются в единый контекст, который подается языковой модели вместе с исходным вопросом.
- Генерация ответа: языковая модель создает ответ, основываясь как на своих внутренних знаниях, так и на предоставленном контексте из найденных документов.
- Интеграция источников: модель может ссылаться на конкретные источники информации, что повышает достоверность и проверяемость ответов.

Компоненты RAG



Шаг 1. Индексирование данных

Шаг 2. Поиск и генерация данных

Типы RAG

Naive RAG (Базовый подход)

- **Простая архитектура:** прямой поиск по векторной базе + генерация ответа без дополнительной обработки
- **Линейный пайплайн:** запрос → embedding → similarity search → топ-k документов → LLM генерация
- **Ограниченный контроль качества:** отсутствие фильтрации и ранжирования найденных документов
- **Быстрая реализация:** минимальные требования к разработке, подходит для MVP и простых сценариев

Advanced RAG (Продвинутый подход)

- **Многоэтапный retrieval:** комбинирование различных стратегий поиска (dense + sparse, гибридный поиск)
- **Реранкинг и фильтрация:** дополнительная обработка найденных документов для улучшения релевантности
- **Chunk optimization:** улучшенные методы сегментации документов и управления размером контекста
- **Query enhancement:** расширение и переформулирование запросов для более точного поиска

Agentic RAG (Агентный подход)

- **Динамическое планирование**: AI-агент самостоятельно определяет стратегию поиска для каждого запроса
- **Итеративный поиск:** многошаговый процесс с возможностью уточнения и дополнительных запросов
- **Tool integration:** использование внешних инструментов и API для получения актуальной информации
- **Self-reflection:** способность анализировать качество найденной информации и корректировать подход

Почему важна оценка RAG систем?

Вызовы современных RAG систем

RAG системы должны не только найти релевантную информацию, но и синтезировать её в точный ответ. Основные проблемы: галлюцинации при неполной информации, зависимость от качества базы знаний, сложность обеспечения фактической точности при генерации.

Многокомпонентность архитектуры усложняет оценку

RAG состоит из цепочки компонентов: векторная база → поиск → ранжирование → генерация. Ошибка в любом звене влияет на результат, но определить источник проблемы сложно. Неточный ответ может быть следствием плохого поиска ИЛИ неправильной интерпретации найденной информации.

Необходимость количественных метрик для улучшения системы

Без систематической оценки невозможно понять, какие компоненты требуют улучшения. Субъективных оценок недостаточно для принятия обоснованных решений. Метрики позволяют сравнивать подходы, оценивать изменения и мониторить качество системы.

Отличия от оценки традиционных ML/DL моделей

RAG требует оценки качественных аспектов: релевантность ответа, корректность интерпретации, полнота информации. Традиционные метрики (BLEU, ROUGE) не подходят, так как не учитывают фактическую точность и практическую полезность ответов.

RAG Triad

Запрос пользователя

ANSWER RELEVANCY
(Актуальность ответа)
Оценивает, насколько ответ LLM релевантен пользовательскому запросу.

CONTEXTUAL RELEVANCY
(Актуальность контекста)
Оценивает, насколько релевантен извлеченный контекст пользовательскому запросу.

Ответ от RAG

Retrieval Context

FAITHFULNESS (ДОСТОВЕРНОСТЬ)
Оценивает, содержит ли ответ LLM галлюцинации на основе извлеченного контекста.

Точность и полнота контекста

Точность контекста (Context Precision):
Определение: насколько хорошо релевантные фрагменты расположены в начале списка результатов

Формула: Precision@k = (Релевантные в позиции i) / i для каждой позиции

Пример расчета:

- Найдено 5 фрагментов: [релевантный, нерелевантный, релевантный, нерелевантный, релевантный]
- Позиция 1: 1/1 = 1.0, Позиция 3: 2/3 = 0.67, Позиция 5: 3/5 = 0.6
- Context Precision = (1.0 + 0.67 + 0.6) / 3 = 0.76

Полнота контекста (Context Recall):
Определение: какая доля информации из эталонного ответа содержится в найденном контексте

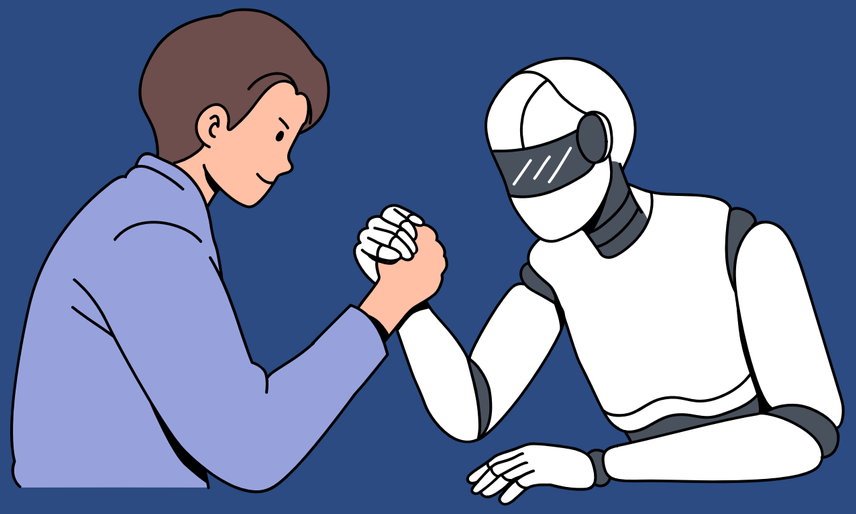
Формула: Поддерживаемые утверждения / Всего утверждений в эталоне

Пример расчета:

- Эталонный ответ содержит 5 фактических утверждений
- В найденном контексте обосновано 4 утверждения
- Context Recall = 4/5 = 0.8

Robustness RAG

НАДЕЖНОСТЬ RAG



Вариации запросов (input-level)

Проверяем, сохраняет ли модель корректность при:

- перефразировках и замене синонимов,
- добавлении шума или опечаток,
- изменении порядка слов,
- частичном вводе или усечённых запросах.

Ожидаемое поведение: результат остаётся инвариантным, смысл ответа тот же.

Вариации контекста (knowledge-level)

Проверяем, как изменяется ответ при изменении базы знаний:

- добавление новых документов,
- удаление или искажение релевантных фрагментов,
- добавление похожих, но не идентичных источников,
- изменение плотности контекста (chunk size, overlap).

Ожидаемое поведение: RAG должен устойчиво выбирать релевантные документы и не “галлюцинировать” при конфликтующих данных.

Как измерять Robustness и Negative checks

Negative checks

В зависимости от формата ответа от RAG системы возможно использовать два типа метрик.

Детерминированные:

- Regex Match, Exact Match, Answer Precision (evaliQA)

Эмбеддинг метрики:

- Semantic Similarity

Robustness

Чаще всего нам требуется сверить ответ с уже неким эталоном, поэтому для оценки robustness достаточно использовать бызове метрики сравнение с ожидаемым результатом.

Детерминированные:

- Answer Precision (evaliQA)

Эмбеддинг метрики:

- Semantic Similarity

Инструменты оценки RAG

ragas

promptfoo

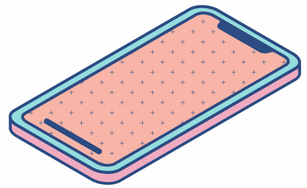
EvaliQA

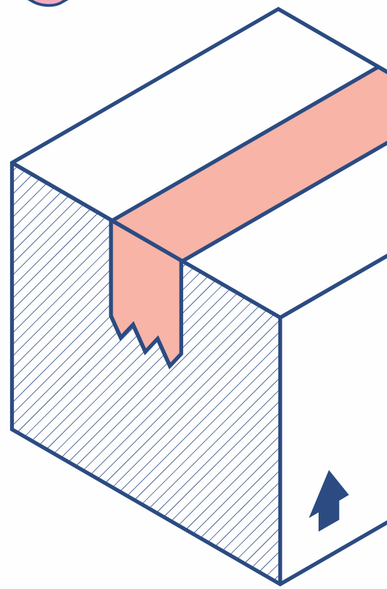
Assess. Assure. Accept

DeepEval.

opik









Практические рекомендации

Поэтапное тестирование

Начинайте с оценки happy path тестов и edge cases, далее переключайтесь на negative тесты и после проверка robustness

Создание тестовых данных

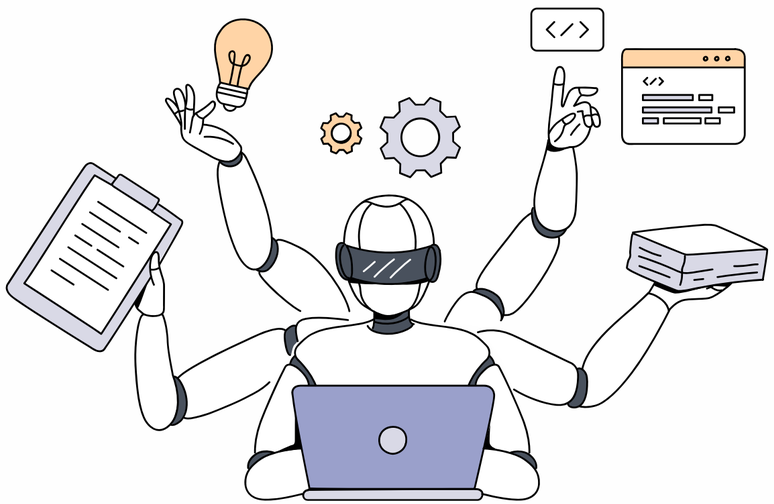
Определите доменные зоны вашей RAG системы, далее определите типы тестов, комбинируйте разные параметры для максимального покрытия оценки поведения

Поэтапная оценка RAG компонентов

Оценивайте отдельно: поиск и генерацию. Это помогает быстро найти проблемные компоненты.

Практика

Оценка RAG



Наши цели

- Изучить, как создать данные для оценки
- Разобрать фреймворки EvaliQA, RAGAS, DeepEval, Promtfoo, Opik
- Выполнить оценку различных метрик и интерпретировать результаты

Делаем самостоятельно

Практика.

- Подготовить датасеты happy path (минимум 50 запросов), negative (мин 20 запросов), metamorphic (мин 3 группы отношений) для оценки RAG.
- Выполнить оценку Answer Relevancy, Context Relevancy и Faithfulness используя любой фреймворк на выбор для happy path данных (можно выполнить на нескольких и сравнить результаты).
- Выполнить оценку Answer Precision или Semantic Similarity для negative и metamorphic тестов. Можно попробвать также использовать Restricted Refusal в EvaliQA для negative тестов.



На сегодня это все!

Вопросы?

